

# DOCUMENT 3

## QUALIFYAI — COMPLETE WEB APP FRONTEND BLUEPRINT

**Project:** QualifyAI

**Document:** Document 3 — Complete Web App Frontend Blueprint

**Document Type:** Frontend / UX / UI Implementation Specification

**Status:** Foundation Frontend Specification

**Version:** 1.0

---

### 1. DOCUMENT OVERVIEW

This document defines the complete frontend blueprint for **QualifyAI**.

Document 1 established the product and functional requirements, while Document 2 established the technical implementation architecture.

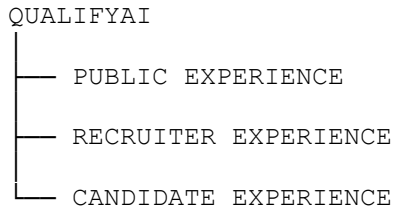
Document 3 translates those requirements into the frontend experience.

It defines:

- frontend application structure,
- public website,
- authentication,
- recruiter experience,
- candidate experience,
- job management,
- JD processing interface,
- rubric interface,
- question management,
- candidate management,
- interview interface,
- real-time voice UI,
- evaluation interface,
- reports,
- analytics,
- proctoring indicators,
- navigation,
- layouts,
- reusable components,
- responsive behavior,

- UX states,
- accessibility,
- and frontend implementation principles.

The frontend must provide two distinct but connected experiences:



The interface must remain consistent with the QualifyAI product identity and technical architecture.

---

## 2. FRONTEND OBJECTIVE

The frontend should transform the technical capabilities of QualifyAI into a simple, understandable, and professional user experience.

The primary frontend objectives are:

1. Make the recruiter workflow easy to understand.
2. Make the candidate interview experience distraction-free.
3. Clearly communicate AI interview states.
4. Make complex evaluation information understandable.
5. Provide recruiters with actionable candidate intelligence.
6. Maintain consistent design across the platform.
7. Provide responsive experiences across supported devices.
8. Keep sensitive information behind appropriate authorization.

---

## 3. FRONTEND TECHNOLOGY

The documented frontend technology direction includes:

- Next.js 14
- React 18
- JavaScript
- Tailwind CSS
- Shadcn UI

The frontend should use reusable React components and a structured domain-oriented architecture.

---

## 4. FRONTEND ARCHITECTURE

The frontend should conceptually follow:

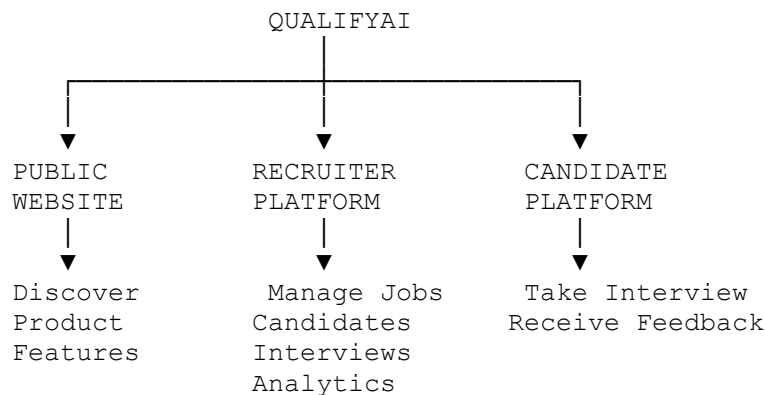
```
client/  
├── public/  
└── src/  
    ├── assets/  
    ├── components/  
    ├── layouts/  
    ├── pages/  
    ├── routes/  
    ├── services/  
    ├── hooks/  
    ├── context/  
    ├── lib/  
    ├── constants/  
    ├── utils/  
    └── styles/
```

The exact implementation may evolve according to the final Next.js architecture, but responsibilities must remain separated.

---

## 5. FRONTEND EXPERIENCE MODEL

QualifyAI should be understood as three major frontend experiences.



---

## 6. PUBLIC EXPERIENCE

The public experience introduces QualifyAI to potential users.

The landing page should communicate:

- what QualifyAI is,
- the recruitment problem,
- the QualifyAI solution,
- AI-powered interviewing,
- Job Description intelligence,
- evaluation,
- integrity,
- analytics,
- candidate feedback,
- and the product's overall value proposition.

The detailed landing-page specification is maintained separately in **Document 6**.

Therefore, this document defines only the application's frontend relationship to the public experience.

---

## 7. PUBLIC ROUTES

The public experience may include:

```
/
├── Home
├── Product
├── Features
├── How It Works
├── For Recruiters
├── For Candidates
├── Technology
└── Authentication Entry
```

The exact route naming can evolve during implementation.

---

## 8. GLOBAL NAVIGATION

The public navigation should provide access to:

- Logo
- Product
- Features
- How It Works
- Recruiter information
- Candidate information
- Login

- Get Started

Authenticated applications should use role-specific navigation.

---

## 9. AUTHENTICATION EXPERIENCE

Authentication is the entry point to the protected application.

The experience should support:

```
Login
↓
Authentication
↓
Identity
↓
Role
↓
Organization / Candidate Context
↓
Appropriate Application
```

Authentication UI should remain simple and focused.

---

## 10. AUTHENTICATION STATES

The frontend should account for:

```
Unauthenticated
↓
Login
↓
Authenticating
↓
Authenticated
↓
Loading Profile
↓
Role Resolution
↓
Application
```

Error states should include:

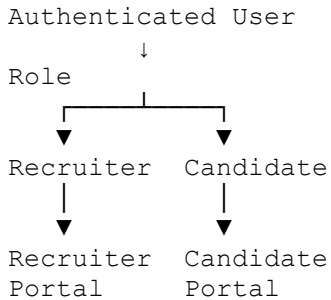
- invalid credentials,
- expired session,
- unauthorized access,

- network error,
- account-related error.

---

## 11. ROLE-BASED FRONTEND

After authentication, the application should determine the user's role.



The frontend should not rely on role checks alone for security.

Backend authorization remains authoritative.

---

## 12. RECRUITER APPLICATION

The recruiter application is the operational center of QualifyAI.

Its primary areas are:



---

## 13. RECRUITER DASHBOARD

The recruiter dashboard should provide an immediate overview of recruitment activity.

Potential information includes:

- Active jobs
- Total candidates
- Pending interviews
- Completed interviews
- Average scores
- Top candidates
- Recent activity

The dashboard should prioritize actionable information over decorative metrics.

---

## 14. DASHBOARD INFORMATION HIERARCHY

A typical recruiter dashboard can follow:

```
Header
↓
Workspace / Organization Context
↓
Summary Metrics
↓
Active Jobs
↓
Candidate Activity
↓
Recent Interviews
↓
Top Candidates
↓
Assessment Insights
```

The exact widgets should be driven by actual product requirements.

---

## 15. RECRUITER SIDEBAR

The recruiter navigation should provide direct access to:

```
Dashboard
Jobs
Candidates
Interviews
Reports
Analytics
Settings
```

Navigation should clearly communicate the current location.

---

## 16. ORGANIZATION CONTEXT

Where organizations are supported, the recruiter interface should clearly indicate the current organization/workspace.

Organization context must not be editable through arbitrary client-side manipulation.

The server remains responsible for validating organization access.

---

## 17. JOB MANAGEMENT

The Jobs section is one of the core recruiter experiences.

The recruiter should be able to:

- view jobs,
- create jobs,
- open job details,
- provide Job Description information,
- inspect requirements,
- generate assessments,
- manage candidates.

Conceptual flow:

```
Jobs
↓
Create Job
↓
Job Information
↓
Job Description
↓
Process JD
↓
Requirements
↓
Rubric
↓
Questions
```

---

## 18. CREATE JOB EXPERIENCE

The job creation interface should collect essential information such as:

- Job title
- Role information
- Job Description
- relevant configuration

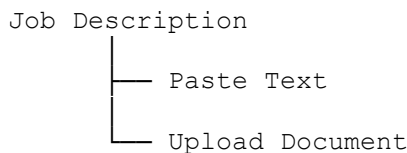
The form should be divided into logical sections rather than presenting a large undifferentiated form.

---

## 19. JOB DESCRIPTION INPUT

The frontend should support the documented JD workflow.

Possible input experience:



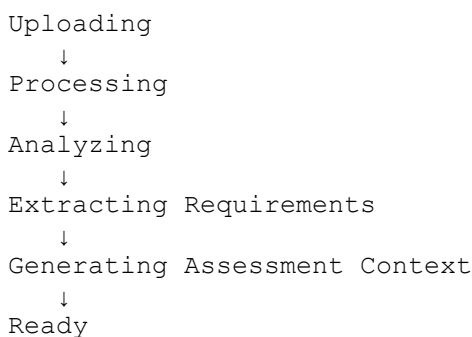
After submission, the user should receive clear processing feedback.

---

## 20. JD PROCESSING UI

The frontend should communicate the processing pipeline.

Example:



The interface should not leave the recruiter wondering whether processing has stopped.

---

## 21. REQUIREMENT REVIEW

After JD processing, recruiters should be able to review extracted requirements.

The interface can group information into:

Role Information  
Required Skills  
Preferred Skills  
Experience  
Responsibilities  
Technical Domains

Where editing is supported, the recruiter should be able to correct extraction errors before assessment generation.

---

## 22. RUBRIC INTERFACE

The rubric interface should allow recruiters to understand how candidates will be evaluated.

A conceptual layout:

Evaluation Rubric

- Technical Competency
- Problem Solving
- Technical Depth
- Communication
- Role-Specific Criteria

Each criterion should communicate:

- what is being evaluated,
  - relative importance where applicable,
  - and how it contributes to the assessment.
- 

## 23. QUESTION MANAGEMENT

The recruiter should be able to inspect the generated question set.

The question interface may display:

Question  
Topic  
Difficulty  
Competency  
Expected Evaluation Area  
Follow-Up Context

Generated questions should remain connected to the job and rubric.

---

## 24. QUESTION REVIEW STATES

The UI should account for:

Generating  
↓  
Generated  
↓  
Reviewing  
↓  
Approved / Configured  
↓  
Assessment Ready

If question editing is supported, modifications must remain associated with the correct job.

---

## 25. CANDIDATE MANAGEMENT

The recruiter candidate interface should provide:

- candidate name,
- job,
- invitation status,
- interview status,
- score,
- completion status,
- ranking,
- report availability.

Conceptually:

Candidates  
↓  
Candidate List  
↓  
Candidate Profile  
↓

Interview  
↓  
Evaluation  
↓  
Report

---

## 26. CANDIDATE LIST

The candidate list should support useful filtering and sorting.

Potential filters:

- Job
- Interview status
- Score range
- Completion state
- Ranking

The interface should avoid unnecessary filters that are not supported by the backend.

---

## 27. CANDIDATE PROFILE

The candidate profile should provide a consolidated view.

Potential sections:

Candidate Information  
↓  
Application / Job  
↓  
Interview Status  
↓  
Overall Score  
↓  
Rubric Performance  
↓  
Communication Metrics  
↓  
Integrity Information  
↓  
Transcript  
↓  
Report

Sensitive information must only be shown to authorized recruiters.

---

## 28. INTERVIEW MANAGEMENT

The Interviews area should provide visibility into:

- upcoming interviews,
- active interviews,
- completed interviews,
- failed/interrupted interviews,
- evaluation status.

Interview status should be visually distinguishable.

---

## 29. INTERVIEW STATUS MODEL

The frontend should support states such as:

Created  
Invited  
Ready  
In Progress  
Processing  
Completed  
Evaluating  
Evaluated  
Report Ready  
Failed

The exact state values should match backend implementation.

---

## 30. CANDIDATE INTERVIEW EXPERIENCE

The candidate interview is the most interaction-sensitive frontend experience.

The screen should prioritize:

- AI interviewer status,
- microphone status,
- current conversation,
- candidate response state,
- connection state,
- interview progress.

Unnecessary navigation should be removed from the active interview environment.

---

# 31. INTERVIEW ROOM

The interview room can conceptually contain:

|                                |                    |          |  |
|--------------------------------|--------------------|----------|--|
| QualifyAI                      | Interview Progress |          |  |
| AI INTERVIEWER                 |                    |          |  |
| Current Question               |                    |          |  |
| AI / Candidate State Indicator |                    |          |  |
| Microphone                     | Connection         | Progress |  |

The actual visual implementation should follow `design.md`.

---

# 32. INTERVIEW PREPARATION SCREEN

Before the interview starts, the candidate should receive:

- interview instructions,
- expected duration where applicable,
- microphone requirements,
- environment guidance,
- integrity/proctoring information,
- start action.

The candidate should clearly understand what will happen before starting.

---

# 33. SYSTEM CHECK

The system should provide a pre-interview readiness experience.

Possible checks include:

Microphone  
↓

Audio Input  
↓  
Connection  
↓  
Browser Compatibility  
↓  
Interview Access

The candidate should not enter the interview until critical requirements are satisfied.

---

## 34. AI INTERVIEWER STATE

The UI should clearly communicate whether the AI is:

Connecting  
Listening  
Thinking / Processing  
Speaking  
Waiting  
Completed

The candidate should never have to guess whether the system is listening or processing.

---

## 35. CANDIDATE SPEAKING STATE

When the candidate is speaking, the interface should provide a clear visual indicator.

For example:

AI  
↓  
Listening...  
↓  
Candidate Voice Activity

Audio visualization may be used, but it should remain functional rather than purely decorative.

---

## 36. AI SPEAKING STATE

When the AI is responding:

AI Speaking  
↓

Audio Output



Candidate Listening

The candidate should not accidentally trigger conflicting interaction states.

---

## 37. PROCESSING STATE

After the candidate finishes speaking, the interface may enter:

Processing Response...

This state is important because the AI pipeline includes:

- transcription,
- reasoning,
- question generation,
- TTS.

The UI should provide enough feedback to prevent users from assuming the system has frozen.

---

## 38. CONNECTION STATE

The interview UI should clearly indicate connection health.

States can include:

Connected  
Connecting  
Weak Connection  
Disconnected  
Reconnecting

If reconnection is possible, the system should communicate progress.

---

## 39. INTERVIEW PROGRESS

The candidate should have an appropriate indication of progress.

However, the UI should avoid exposing information that could compromise adaptive assessment.

Depending on the final interview design, progress may be represented as:

- elapsed time,
- interview stage,
- general progress indicator.

Exact question counts should only be displayed if compatible with the assessment design.

---

## 40. INTERVIEW COMPLETION

At the end of the interview, the candidate should receive a clear completion state.

```
Interview Complete
      ↓
Response Processing
      ↓
Evaluation
      ↓
Feedback Available
```

The UI should not imply that a final evaluation is available until the backend confirms it.

---

## 41. CANDIDATE FEEDBACK

The candidate feedback interface should provide meaningful information.

Potential sections:

```
Overall Performance
Technical Strengths
Technical Improvement Areas
Communication Insights
Assessment Summary
```

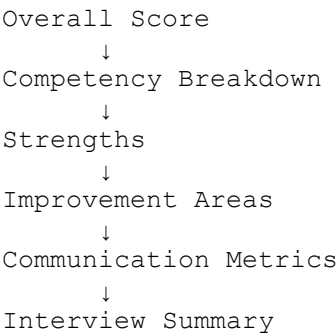
The candidate should not receive recruiter-only information.

---

## 42. DIAGNOSTIC REPORT

The diagnostic report interface should make the evaluation understandable.

A conceptual hierarchy:



The report should prioritize clarity over excessive visual complexity.

---

## 43. RECRUITER EVALUATION VIEW

Recruiters require more detailed information than candidates.

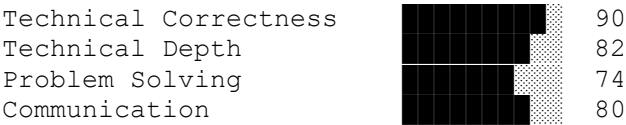
The recruiter evaluation view may include:

- overall score,
  - rubric scores,
  - technical evaluation,
  - communication metrics,
  - transcript,
  - integrity signals,
  - report,
  - candidate ranking.
- 

## 44. RUBRIC SCORE VISUALIZATION

The frontend should visually represent rubric performance.

For example:



The actual visualization should follow the design system.

---

## 45. TRANSCRIPT VIEWER

The transcript interface should distinguish between:

AI  
Candidate

A conversation-style presentation is appropriate.

Example:

AI  
Question...

Candidate  
Response...

AI  
Follow-up...

Candidate  
Response...

Transcript navigation should support long interviews without creating excessive scrolling difficulty.

---

## 46. TRANSCRIPT METADATA

Where available and appropriate, transcript segments may include:

- timestamp,
- speaker,
- turn,
- question association.

The interface should avoid exposing internal AI reasoning or hidden chain-of-thought.

---

## 47. PROCTORING VIEW

Recruiters may receive a summarized integrity view.

Possible information includes:

Integrity Status  
↓

Detected Events  
↓  
Event Timeline  
↓  
Risk / Trust Information

The UI should distinguish actual recorded events from generated interpretations.

---

## 48. PROCTORING EVENT TIMELINE

Events can be displayed chronologically.

Example categories:

- Focus lost
- Tab switch
- Browser visibility change
- Acoustic anomaly

The exact event taxonomy should follow the backend implementation.

---

## 49. CANDIDATE RANKING

The recruiter ranking interface should allow comparison of candidates for a specific job.

Conceptually:

Rank  
Candidate  
Overall Score  
Technical Score  
Communication  
Integrity  
Status

Ranking must always remain contextual to the selected job.

---

## 50. ANALYTICS DASHBOARD

The recruiter analytics interface may include:

- candidate score distribution,
- average score,
- completion rate,
- interview status,
- competency trends,
- candidate comparisons.

Charts should communicate information rather than simply decorate the dashboard.

---

## 51. REPORTS AREA

The Reports section should allow authorized recruiters to access generated reports.

Possible states:

Generating  
Processing  
Ready  
Failed

A report should not be shown as ready before generation is confirmed.

---

## 52. GLOBAL COMPONENT SYSTEM

The frontend should maintain reusable components.

Core shared components may include:

Buttons  
Inputs  
Forms  
Modals  
Dialogs  
Dropdowns  
Tabs  
Tables  
Cards  
Badges  
Alerts  
Toast  
Loading States  
Empty States  
Error States  
Pagination  
Charts  
Navigation

Domain-specific components should remain inside their respective domain boundaries.

---

## 53. DESIGN SYSTEM

The frontend should follow the rules established in:

`design.md`

The design system should define:

- colors,
- typography,
- spacing,
- radii,
- shadows,
- component styles,
- interaction states,
- responsive behavior.

Components should not introduce arbitrary visual styles.

---

## 54. RESPONSIVE DESIGN

QualifyAI should support responsive experiences.

Important contexts include:

- desktop recruiter dashboard,
- laptop candidate interview,
- tablet where applicable,
- mobile public website.

The active interview experience should prioritize supported screen sizes appropriate to the final interview requirements.

---

## 55. ACCESSIBILITY

The frontend should account for:

- keyboard navigation,
- semantic HTML,
- accessible labels,
- sufficient contrast,
- visible focus states,
- readable typography,
- appropriate error messages.

Voice interaction should not be the only mechanism for communicating important system information.

---

## 56. LOADING STATES

Every asynchronous operation should have an appropriate loading state.

Examples:

```
Loading Dashboard
Processing JD
Generating Rubric
Generating Questions
Loading Candidates
Connecting Interview
Processing Response
Generating Report
```

Avoid blank screens during asynchronous operations.

---

## 57. EMPTY STATES

Empty states should explain what the user can do next.

Example:

```
No Jobs Yet

Create your first job to start
building a technical assessment.
```

Empty states should be actionable where appropriate.

---

## 58. ERROR STATES

Errors should be:

- clear,
- concise,
- actionable.

Examples:

Unable to process Job Description.

Try again or review the uploaded document.

Do not expose internal server errors or provider credentials.

---

## 59. SUCCESS STATES

Important actions should provide confirmation.

Examples:

- Job created
- JD processed
- Rubric generated
- Candidates invited
- Interview completed
- Report generated

Feedback should be immediate but not intrusive.

---

## 60. NOTIFICATION SYSTEM

The frontend can use non-blocking notifications for:

- successful actions,
- warnings,
- failures,
- background task completion.

Critical interview errors should use stronger visual treatment than ordinary notifications.

---

## 61. ROUTE PROTECTION

Protected routes should verify authentication state.

Conceptually:

```
Route
↓
Authentication
↓
Role
↓
Organization / Resource Access
↓
Page
```

Frontend route protection improves UX but does not replace backend authorization.

---

## 62. FRONTEND API LAYER

The client should communicate with the backend through a centralized service/API layer.

Avoid scattering raw API requests across arbitrary components.

Conceptually:

```
Component
↓
Hook / Service
↓
API Client
↓
Backend
```

This improves maintainability and consistency.

---

## 63. FRONTEND STATE MANAGEMENT

State should be separated by responsibility.

Potential categories:

**Local UI State**

- modal visibility,
- selected tab,
- form state.

## Application State

- authenticated user,
- organization context,
- notifications.

## Server State

- jobs,
- candidates,
- interviews,
- evaluations.

## Real-Time State

- connection status,
- audio status,
- interview status,
- current conversation turn.

Do not put all application data into one global state store unnecessarily.

---

# 64. REAL-TIME FRONTEND STATE

The interview UI requires synchronized state.

Conceptually:

```

WebSocket
  ↓
Real-Time Event
  ↓
Interview State
  ↓
UI
  
```

Events may include:

- connected,
- interview started,
- AI speaking,
- candidate speaking,

- transcript received,
  - processing,
  - next question,
  - interview completed,
  - error,
  - reconnecting.
- 

## 65. AUDIO UI

The candidate interview interface should communicate microphone state.

Possible states:

```
Microphone Ready  
Microphone Active  
Microphone Muted  
Microphone Permission Required  
Microphone Error
```

The system should request browser permissions at an appropriate time.

---

## 66. SECURITY IN FRONTEND

The frontend must not contain:

- private API keys,
- service-role credentials,
- AI provider secrets,
- database credentials.

Only public/client-safe configuration may be exposed.

Sensitive data must be requested from authorized backend endpoints.

---

## 67. PERFORMANCE OPTIMIZATION

The frontend should optimize:

- initial load,
- route transitions,

- dashboard rendering,
- large candidate lists,
- transcript rendering,
- chart rendering,
- interview audio interaction.

Long transcripts and candidate datasets should not unnecessarily render thousands of elements simultaneously.

---

## 68. SEO

The public website should support appropriate SEO.

Authenticated dashboards and candidate interview pages generally have different requirements from public marketing pages.

The landing page should provide:

- meaningful page titles,
  - metadata,
  - semantic structure,
  - appropriate headings,
  - descriptive content.
- 

## 69. FRONTEND ANALYTICS

Where product analytics are implemented, the system may track high-level product events such as:

- signup,
- job creation,
- JD processing,
- candidate invitation,
- interview start,
- interview completion.

Do not collect unnecessary sensitive interview content merely for analytics.

---

## 70. FRONTEND DIRECTORY BLUEPRINT

The frontend architecture can be represented as:

```
client/
├── public/
├── src/
│   ├── assets/
│   ├── components/
│   │   ├── common/
│   │   ├── auth/
│   │   ├── recruiter/
│   │   ├── candidate/
│   │   ├── jobs/
│   │   ├── interviews/
│   │   ├── evaluation/
│   │   ├── reports/
│   │   ├── analytics/
│   │   └── proctoring/
│   ├── layouts/
│   ├── pages/
│   │   ├── public/
│   │   ├── auth/
│   │   ├── recruiter/
│   │   └── candidate/
│   ├── routes/
│   ├── services/
│   ├── hooks/
│   ├── context/
│   ├── lib/
│   ├── constants/
│   ├── utils/
│   └── styles/
```

This is a logical blueprint; actual Next.js routing may adapt this structure according to the final implementation approach.

---

## 71. PUBLIC → AUTHENTICATED FLOW

```
Landing Page
  ↓
Get Started
  ↓
Authentication
  ↓
Role Resolution
  ↓
Recruiter / Candidate Portal
```

---

## 72. RECRUITER FRONTEND FLOW

Dashboard  
↓  
Jobs  
↓  
Create Job  
↓  
JD Processing  
↓  
Requirements  
↓  
Rubric  
↓  
Questions  
↓  
Candidates  
↓  
Invite  
↓  
Interviews  
↓  
Evaluation  
↓  
Ranking  
↓  
Reports  
↓  
Analytics

---

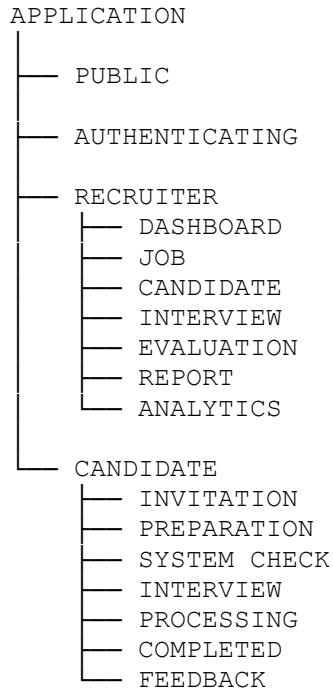
## 73. CANDIDATE FRONTEND FLOW

Invitation  
↓  
Authentication / Access  
↓  
Instructions  
↓  
System Check  
↓  
Interview Room  
↓  
AI Conversation  
↓  
Interview Complete  
↓  
Evaluation  
↓  
Diagnostic Feedback

---

## 74. COMPLETE FRONTEND STATE MODEL

The major frontend states can be represented as:



## 75. FRONTEND UX PRINCIPLES

QualifyAI should follow these principles:

### Clarity

Users should understand what is happening.

### Focus

The candidate interview interface should minimize distraction.

### Consistency

Similar actions should behave similarly throughout the platform.

### Feedback

System actions should provide visible feedback.

### Trust

AI-driven decisions should be presented carefully and transparently.

### **Accessibility**

Important information should remain accessible regardless of interaction method.

### **Responsiveness**

Interfaces should adapt appropriately to supported screen sizes.

---

## **76. FRONTEND SECURITY PRINCIPLES**

The frontend must:

- never trust client-only authorization,
  - never expose private secrets,
  - validate user interaction where appropriate,
  - avoid exposing unauthorized data,
  - handle expired sessions,
  - clear sensitive state appropriately,
  - use secure communication.
- 

## **77. FRONTEND ACCEPTANCE CRITERIA**

The frontend blueprint is considered successfully implemented when:

### **Public**

- Users can understand the QualifyAI product.
- Public navigation works.
- Authentication entry points are clear.

### **Recruiter**

- Recruiters can navigate their workspace.
- Jobs can be created.
- JD processing is understandable.
- Requirements are visible.
- Rubric is visible.
- Questions are manageable.
- Candidates are manageable.
- Interviews are visible.

- Evaluation is understandable.
- Reports are accessible.
- Candidate ranking is usable.
- Analytics are understandable.

## Candidate

- Invitations can be accessed.
- Instructions are clear.
- System readiness can be checked.
- Voice interview states are visible.
- Candidate knows when the AI is speaking/listening/processing.
- Interview completion is clear.
- Diagnostic feedback is understandable.

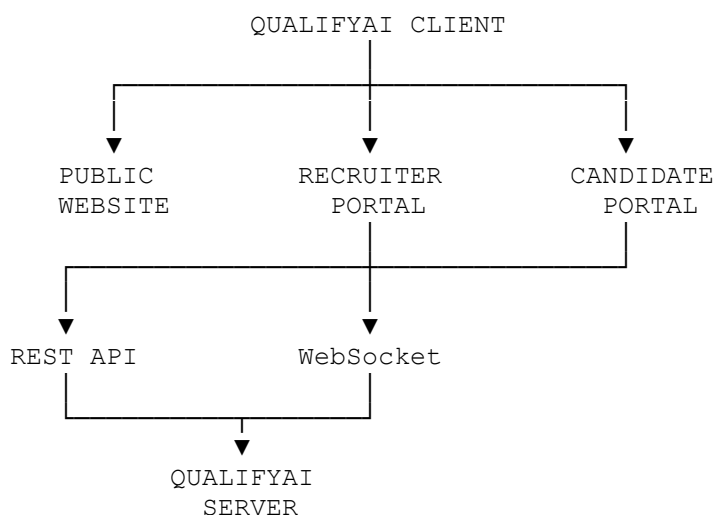
## Platform

- Protected routes are enforced.
- Responsive behavior is implemented.
- Accessibility is considered.
- Loading, empty, error, and success states exist.
- Real-time state is synchronized.
- Sensitive data is not unnecessarily exposed.

---

# 78. FINAL FRONTEND ARCHITECTURE

The complete frontend relationship is:



The frontend therefore acts as the presentation and interaction layer over the QualifyAI backend while maintaining specialized experiences for recruiters and candidates.

---

## 79. FINAL FRONTEND DEFINITION

The QualifyAI frontend is not simply a collection of dashboards.

It consists of three connected experiences:

**A public product experience, an intelligence-driven recruiter workspace, and a focused conversational candidate interview environment.**

The core frontend journey is:

```
DISCOVER
  ↓
AUTHENTICATE
  ↓
CONFIGURE
  ↓
ASSESS
  ↓
INTERVIEW
  ↓
EVALUATE
  ↓
ANALYZE
  ↓
DECIDE
  ↓
LEARN
```

The recruiter uses the platform to **configure and analyze technical assessments**.

The candidate uses the platform to **participate in an adaptive AI interview**.

The frontend connects these two experiences into one consistent QualifyAI product.

---

## DOCUMENT 3 — END

**Document:** QualifyAI — Complete Web App Frontend Blueprint

**Status:** Foundation Frontend Specification

**Purpose:** Defines the complete web application frontend experience and structure

**Previous:** Document 1 — Product & Functional Blueprint

**Previous:** Document 2 — Full Technical Implementation Plan

**Next:** Document 4 — QualifyAI Software Architecture & Development Structure